

Write a program to simulate deadlock detection.

```
#include <stdio.h>
#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int allocation[MAX_PROCESSES][MAX_RESOURCES];
int request[MAX_PROCESSES][MAX_RESOURCES];
int available[MAX_RESOURCES];
int resources[MAX_RESOURCES];
int work[MAX_RESOURCES];
int marked[MAX_PROCESSES];

int main() {
    int num_processes, num_resources;

    printf("Enter the number of processes: ");
    scanf("%d", &num_processes);

    printf("Enter the number of resources: ");
    scanf("%d", &num_resources);

    // Input total resources
    for (int i = 0; i < num_resources; i++) {
        printf("Enter the total amount of Resource R%d: ", i + 1);
        scanf("%d", &resources[i]);
    }

    // Input request matrix
    printf("Enter the request matrix:\n");
    for (int i = 0; i < num_processes; i++) {
        for (int j = 0; j < num_resources; j++) {
            scanf("%d", &request[i][j]);
        }
    }

    // User Input allocation matrix
    printf("Enter the allocation matrix:\n");
```

```

for (int i = 0; i < num_processes; i++) {
    for (int j = 0; j < num_resources; j++) {
        scanf("%d", &allocation[i][j]);
    }
}

// Initialization of the available resources
for (int j = 0; j < num_resources; j++) {
    available[j] = resources[j];
    for (int i = 0; i < num_processes; i++) {
        available[j] -= allocation[i][j];
    }
}

// Mark processes with zero allocation
for (int i = 0; i < num_processes; i++) {
    int count = 0;
    for (int j = 0; j < num_resources; j++) {
        if (allocation[i][j] == 0) {
            count++;
        } else {
            break;
        }
    }
    if (count == num_resources) {
        marked[i] = 1;
    }
}

// Initialize work with available
for (int j = 0; j < num_resources; j++) {
    work[j] = available[j];
}

// Mark processes with requests <= work
for (int i = 0; i < num_processes; i++) {
    int can_be_processed = 1;
    if (marked[i] != 1) {

```

```

for (int j = 0; j < num_resources; j++) {
    if (request[i][j] > work[j]) {
        can_be_processed = 0;
        break;
    }
}
if (can_be_processed) {
    marked[i] = 1;
    for (int j = 0; j < num_resources; j++) {
        work[j] += allocation[i][j];
    }
}
}
}

// Check for unmarked processes (deadlock)
int deadlock = 0;
for (int i = 0; i < num_processes; i++) {
    if (marked[i] != 1) {
        deadlock = 1;
        break;
    }
}

if (deadlock) {
    printf("Deadlock detected\n");
} else {
    printf("No deadlock possible\n");
}

return 0;
}

```

Output:

*Enter the number of processes: 3*

*Enter the number of resources: 3*

*Enter the total amount of Resource R1: 4*

*Enter the total amount of Resource R2: 5*

*Enter the total amount of Resource R3: 7*

*Enter the request matrix:*

1 2 3

4 5 6

7 8 9

*Enter the allocation matrix:*

5 6 7 1 2 3 7 8 9

*Deadlock detected*